1 Lesson 06-11-26

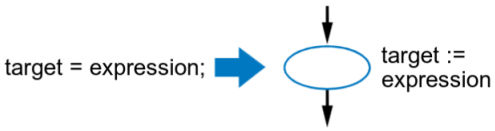
1.1 From C to HLSM

In the last lesson we introduced the SAD calculator in order to demonstrate that sometimes it is useful to give *algorithmic* descriptions.

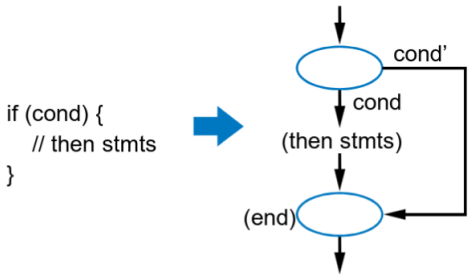
Today we wish to expand on this, showing that a large subset of high level languages such as C can be converted to an HLSM (which then gets synthesized in some other way like datapath & controller). This is also known as a **single-purpose processor**.

The idea is to convert each C construct to equivalent states and transitions:

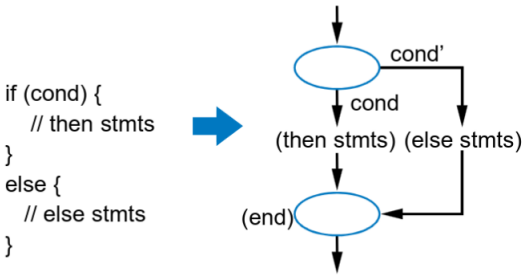
- **Assignment statement** becomes one state with assignment;



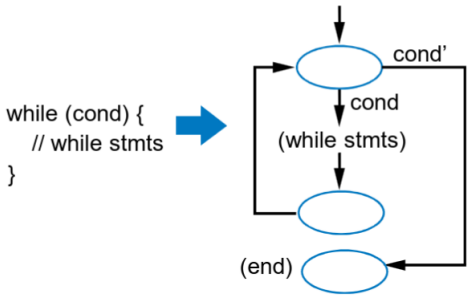
- **If-then statement** becomes state with condition check, transitioning to "then" statements if condition true, otherwise to ending state. "then" statements would also be converted to states;



- **If-then-else** Becomes state with condition check, transitioning to "then" statements if condition true, or to "else" statements if condition false;



- **While loop statement** Becomes state with condition check, transitioning to while loop's statements if true, then transitioning back to condition check.



So we can:

1. Convert each construct to states;
2. Simplify, e.g., merge states;
3. Use RTL design process to convert this into an actual circuit.

We can thus convert C to a circuit using a fairly straightforward process. Actually, a full subset of C is available(though not all C constructs are easily convertible).

Some guidelines for the RTL synthesis:

- Create a register for any declared variable;
- Create a functional unit for each arithmetic operation
- Connect the ports, registers and functional units based on reads and writes in the "code". Use multiplexors for multiple sources;
- Create a unique identifier for each datapath component control input and output.

The rest of the synthesis (controller) follows from the HLSM.

1.1.1 Optimizing single-purpose processors

Optimization is the task of making design metric values the best possible.

Areas of possible improvements in the **HLSM**:

- Merge states: states with constants on transitions can be eliminated, transition taken is already known states with independent operations can be merged;
- Separate states: states which require complex operations can be broken into smaller states to reduce hardware size;
- State encoding: the task of assigning a unique bit pattern to each state in an FSM. Size of state register and combinational logic vary, can be treated as an ordering problem;
- State minimization: the task of merging equivalent states into a single state. States are equivalent iff for all possible input combinations they generate the same outputs and transitions to the same or equivalent next states.

The **datapath** can also benefit from some fucking around:

- Sharing of functional units one-to-one mapping, as done previously, is not necessary. If same operation occurs in different states, they can share a single functional unit;
- Multi-functional units: an ALU, for instance, supports a variety of operations, it can be shared among operations occurring in different states. Basically, we're getting close to an actual processor architecture.

1.2 Serial-parallel multiplicator

Let's look at a multiplier for 2 positive integers `X_in` and `Y_in` on n bits, to $2n$ bits.. The result `Z_out` belongs to the range $[0, 2^n - 1]$. Start signals availability of inputs. Done signals availability of result.

The idea is to use the recurrence relation:

$$\begin{cases} Z_0 = 0 \\ Z_{i+1} = \frac{1}{2}\left(Z_i + X \cdot 2^n \cdot Y_i\right) \end{cases}$$

The result will simply be Z_n.

We see that the primitives used are simple:

- Multiplication of X times 2^n: left-shift of X, n times, inserting 0s;
- multiplication by Y_i: either 0 or $X \cdot 2^n$;
- Sum: simple adder;
- Division by 2: right-shift by 1 position.

Note that one bit of Y is used at a time (serial) as well as all bits of X (parallel): from this we get the name **serial-parallel algorithm**.

1.3 Memory components

RTL design instantiates datapath components to create datapath, controlled by a controller. Some components are used outside the controller and DP.

One of these is **memory**.

1.3.1 Register files

We already saw **register files** in 4.4.

1.3.2 Random access memories

This is an alternative to register files, cheaper but slower.

RAM, that is *Random access memory*, named like this several decades ago to contrast with sequentially-accessed storage like tape drives.

It is logically same as register file—Memory with address inputs, data inputs/outputs, and control, though RAM usually has one port; RF usually two or more.

Let's quickly compare RAM and register files:

- RAM typically larger than about 512 or 1024 words;

- RAM typically stores bits using a bit storage approach that is more efficient than a flip-flop;
- RAM is typically implemented on a chip in a square rather than rectangular shape—keeps longest wires (hence delay) short.